

Multi-Head Attention

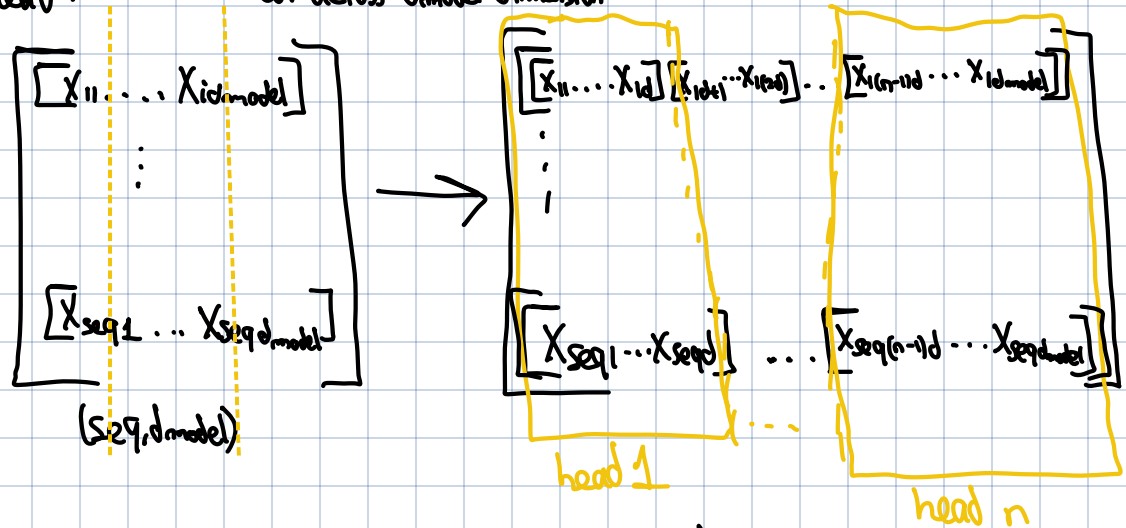
$$\text{Attn}(Q, K, V) = \text{Softmax} \left(\overbrace{\frac{QK^T}{\sqrt{d_k}}}^{(seq, seq)} \right) V$$

$$\text{head}_i = \text{Attn}(XW_i^Q, XW_i^K, XW_i^V)$$

$$X: (seq, d_{model})$$

Multihead:

→ Cut across d_{model} dimension



$$(seq, \frac{d_{model}}{d}, d)$$

↳ d is also called d_{head}

Online Softmax

• Original Softmax:

Pseudocode:

$m_0 = -\infty$
for $i = 1$ to N ①
 $m_i = \max(m_{i-1}, x_i)$

$l_0 = 0$
for $j = 1$ to N ②
 $l_j = l_{j-1} + e^{x_j - m_N}$

for $k = 1$ to N ③
 $x_k \leftarrow \frac{e^{x_k - m_N}}{l_N}$

Runtime: $O(N) \rightarrow 3$ different loops

• We can fuse loops ① and ② by calculating m_i and l_i together, replacing m_N with m_i and adding a correction factor for l_{i-1} :

$$l_i = l_{i-1} \cdot (e^{m_{i-1} - m_i} + e^{x_i - m_i})$$

$$\text{E.g. } l_2 = (e^{3-3} + e^{2-3}) \cdot e^{3-5} + e^{5-5} \quad (x_i = 5, m_i = 5)$$
$$= e^{3-5} + e^{2-5} + e^{5-5}$$

Can prove that it works by induction!

$$m_0 = -\infty$$

$$l_0 = 0$$

for $i = 1$ to N

$$m_i = \max(m_{i-1}, x_i)$$

$$l_i = l_{i-1} \cdot e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

for $k = 1$ to N

$$x_k \leftarrow \frac{e^{x_k - m_N}}{l_N}$$

Block Matrix Multiplication

• Matrix multiplication:

$$\boxed{A} \times \boxed{B} = \boxed{C}$$

$(M, k) \quad (k, N) \quad (M, N)$

Perfect parallelization: If we have $M \times N$ cores (unrealistic)

Solution: Divide Matrix into blocks !!

E.g.

A_{11}	A_{12}	$\times$	B_{11}	B_{12}	B_{13}	B_{14}	$=$	$A_{11}B_{11}$ + $A_{12}B_{21}$	$A_{11}B_{12}$ + $A_{12}B_{22}$	$A_{11}B_{13}$ + $A_{12}B_{23}$	$A_{11}B_{14}$ + $A_{12}B_{24}$
A_{21}	A_{22}		B_{21}	B_{22}	B_{23}	B_{24}		$A_{21}B_{11}$ + $A_{22}B_{21}$	$A_{21}B_{12}$ + $A_{22}B_{22}$	$A_{21}B_{13}$ + $A_{22}B_{23}$	$A_{21}B_{14}$ + $A_{22}B_{24}$

Original: (8, 4)

Block: (2, 2)

A_{ij}

(4, 8)

Block: (2, 2)

B_{ij}

(8, 8)

Block: (2, 2)

Attention:

$Q_1 \left\{ \begin{array}{|c|} \hline 1 \dots d \\ \hline 1 \dots d \\ \hline \vdots \\ \hline \end{array} \right.$

$\rightarrow Q_i \text{ has } l \text{ rows}$

Q_n

$\begin{array}{|c|c|c|c|} \hline k_1^T & k_2^T & & k_n^T \\ \hline 1 \dots & 1 \dots & & 1 \dots \\ \hline \vdots & \vdots & & \vdots \\ \hline d \dots & d \dots & & d \dots \\ \hline \end{array}$

$\dots$

$= S$

$Q: (N, d)$

Block: (l, d)

$K^T: (d, N)$

Block: (d, l)

where $l = \frac{N}{n}$

$$S = \begin{array}{|c|c|c|c|} \hline Q_1 K_1^T & \dots & Q_1 K_n^T \\ \hline \vdots & \ddots & \vdots \\ \hline Q_n K_1^T & \dots & Q_n K_n^T \\ \hline \end{array} \quad (N, N) \quad N = \text{seq_len} - Q/K$$

← Skipping Softmax for now

SV:
$$\begin{bmatrix} Q_{11}K_1^T & \dots & Q_{1n}K_n^T \\ \vdots & \ddots & \vdots \\ Q_{n1}K_1^T & \dots & Q_{nn}K_n^T \end{bmatrix} \begin{bmatrix} V_1 \\ \vdots \\ V_n \end{bmatrix} = 0$$

$S: (N, N)$
Block: (l, l)
 $V: (N, d)$
Block: (l, d)
 $l = \frac{N}{n}$

$\rightarrow (l, l) \cdot (l, d) = (l, d) \rightarrow l \text{ rows, } d \text{ columns}$

$$\bigcirc = \begin{bmatrix} Q_{11}K_1^T V_1 + Q_{12}K_2^T V_2 + \dots + Q_{1n}K_n^T V_n \\ \vdots \\ Q_{n1}K_1^T V_1 + Q_{n2}K_2^T V_2 + \dots + Q_{nn}K_n^T V_n \end{bmatrix} \rightarrow (l \cdot n, d) = (N, d)$$

With Softmax*:

$\rightarrow S_{ij} = \begin{bmatrix} a_{i1} & \dots & a_{i2} \\ \vdots & \ddots & \vdots \\ a_{j1} & \dots & a_{j2} \end{bmatrix} \xrightarrow{\text{Softmax}^*} \begin{bmatrix} \exp(a_{i1} - \max(a_{i1}, \dots, a_{i2})) \\ \vdots \\ \exp(a_{j1} - \max(a_{j1}, \dots, a_{j2})) \end{bmatrix}$

$\rightarrow (l, l) = (l, d) \times (d, l)$

$$S = \begin{bmatrix} Q_{11}K_1^T & \dots & Q_{1n}K_n^T \\ \vdots & \ddots & \vdots \\ Q_{n1}K_1^T & \dots & Q_{nn}K_n^T \end{bmatrix} \xrightarrow{\text{Softmax}^*} \begin{bmatrix} P_{11} & \dots & P_{1n} \\ \vdots & \ddots & \vdots \\ P_{n1} & \dots & P_{nn} \end{bmatrix} = P$$

(N, N)

$P_{ij} = \text{Softmax}^*(Q_{ik}K_j^T)$

$(\text{Softmax}^*(x_i) = \exp(x_i - x_{\max}))$
↳ No normalization

$\text{Softmax}^*(S_{ij}) = \exp(S_{ij} - \text{rowmax}(S_i))$
↳ Scalar ↳ row in the block

→ This is wrong because we use row max of the block row, not the entire matrix row.

Thus, We cannot have $PV = \begin{bmatrix} P_{11}V_{11} + \dots + P_{1n}V_n \\ \vdots \\ P_{n1}V_{11} + \dots + P_{nn}V_n \end{bmatrix} = \begin{bmatrix} 0_1 \\ \vdots \\ 0_n \end{bmatrix}$

Instead, we use Online Softmax :

$$m_0 = -\infty$$

$$l_0 = 0$$

for $i = 1$ to N

$$m_i = \max(m_{i-1}, x_i)$$

$$l_i = l_{i-1} \cdot e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

for $k = 1$ to N

$$x_k \leftarrow \frac{e^{x_k - m_N}}{l_N}$$

For Matrices:

Step 1:

$$m_0 = \begin{bmatrix} -\infty \\ \vdots \\ -\infty \end{bmatrix}_l$$

$$l_0 = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}_l$$

$$O_0 = \begin{bmatrix} 0 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 0 \end{bmatrix}_{l \times d}$$

→ This is the output of the set of blocks $Q_i k_i^T$ ($1 \leq i \leq N$)

$$m_1 = \max(\text{rowmax}(Q_1 k_1^T), m_0)$$

$$S_1 = Q_1 k_1^T$$

→ S_1 is $(l, 1)$, m_1 is $(l, 1)$, so apply $-m_1$ to each column

$$L_1 = \text{row_sum}(\exp(S_1 - m_1)) + l_0 \cdot \exp(m_0 - m_1)$$

$$P_{11} = \exp(S_1 - m_1)$$

$$O_1 = \text{diag}(\exp(m_0 - m_1)) O_0 + \overbrace{P_{11} V_1}^{(l, 1) \times (1, d) = (l, d)}$$

Step 2:

$$m_2 = \max(\text{rowmax}(Q_2 k_2^T), m_1)$$

$$S_2 = Q_2 k_2^T$$

$$L_2 = \text{row_sum}(\exp(S_2 - m_2)) + l_0 \cdot \exp(m_1 - m_2)$$

$$P_{12} = \exp(S_1 - m_2)$$

$$O_2 = \text{diag}(\exp(m_1 - m_2)) O_1 + P_{12} V_2$$

And so on from 1 to n . Then do the same for each Q_i $1 \leq i \leq n$

A more in-depth look at O_1 :

$$\begin{aligned} O_1 &= \text{diag}(\exp(m_1 - m_2)) O_0 \\ &= \begin{bmatrix} \exp(m_1 - m_2)_1 & & \\ & \ddots & \\ 0 & & \exp(m_1 - m_2)_l \end{bmatrix} \begin{bmatrix} O_{11} & \dots & O_{1d} \\ \vdots & & \vdots \\ O_{l1} & \dots & O_{ld} \end{bmatrix} \\ &= \begin{bmatrix} O_{11} \cdot \exp(m_1 - m_2)_1 & \dots & O_{1d} \cdot \exp(m_1 - m_2)_1 \\ \vdots & \ddots & \vdots \\ O_{l1} \cdot \exp(m_1 - m_2)_l & \dots & O_{ld} \cdot \exp(m_1 - m_2)_l \end{bmatrix} \end{aligned} \rightarrow \text{Adjusting each row with the new max}$$

Normalization Factor: (We need $\ln$)

Step $n+1$:

$$\begin{aligned} O_{n+1} &= [\text{diag}(\ln)]^{-1} O_n \\ &= \begin{bmatrix} \frac{1}{\ln^{(1)}} & & 0 \\ & \ddots & \\ 0 & & \frac{1}{\ln^{(n)}} \end{bmatrix} O_n \end{aligned}$$

Note: O_{n+1} in this case is the normalized output for the first l rows of the final matrix. You have to do this for all Q_i 's ($1 \leq i \leq n$)

Algorithm 1 FLASHATTENTION-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ of size $B_c \times d$ each. ↗ n
→ l for us
- 2: Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each. ↘ l for us
- 3: **for** $1 \leq i \leq T_r$ **do** ← **FOR EACH $\mathbf{Q}_i$ BLOCK**
- 4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
- 5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
- 6: **for** $1 \leq j \leq T_c$ **do** ← **FOR EACH $\mathbf{K}_j$ BLOCK**
- 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 9: On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$
 (pointwise), $\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
- 10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
- 11: **end for**
- 12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
- 13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
- 14: Write $\mathbf{O}_i$ to HBM as the i -th block of $\mathbf{O}$.
- 15: Write L_i to HBM as the i -th block of L .
- 16: **end for**
- 17: Return the output $\mathbf{O}$ and the logsumexp L .

Intro to CUDA

Vector Addition:

$$\vec{A} + \vec{B} = \vec{C}$$

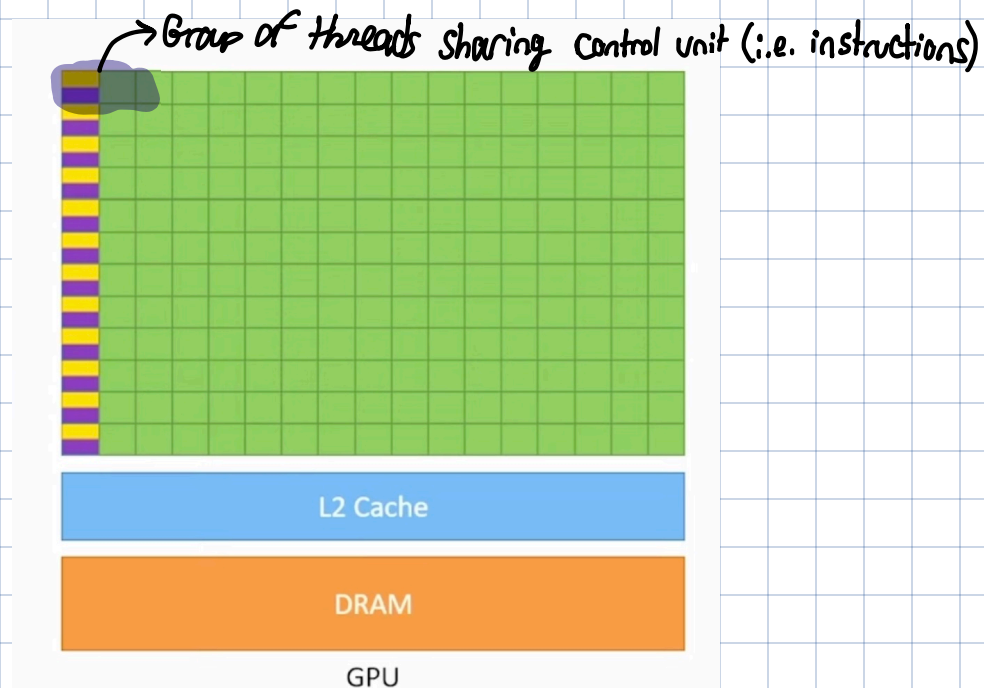
$$A = [a_1 | a_2 | a_3 | a_4 | a_5]$$

$$+ \\ B = [b_1 | b_2 | b_3 | b_4 | b_5]$$

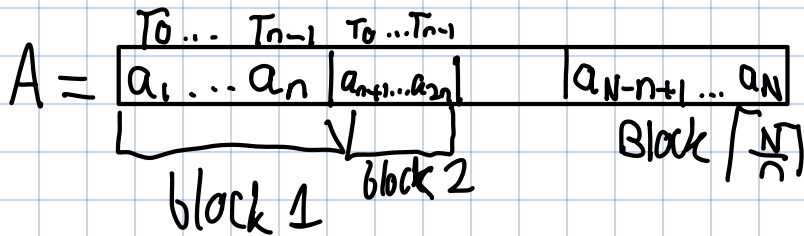
$$C = [a_1+b_1 | a_2+b_2 | a_3+b_3 | a_4+b_4 | a_5+b_5]$$

$T_0 \quad T_1 \quad T_2 \quad T_3 \quad T_4$

A group of threads, called a **warp**, shares the same control units



Vector addition with blocks:



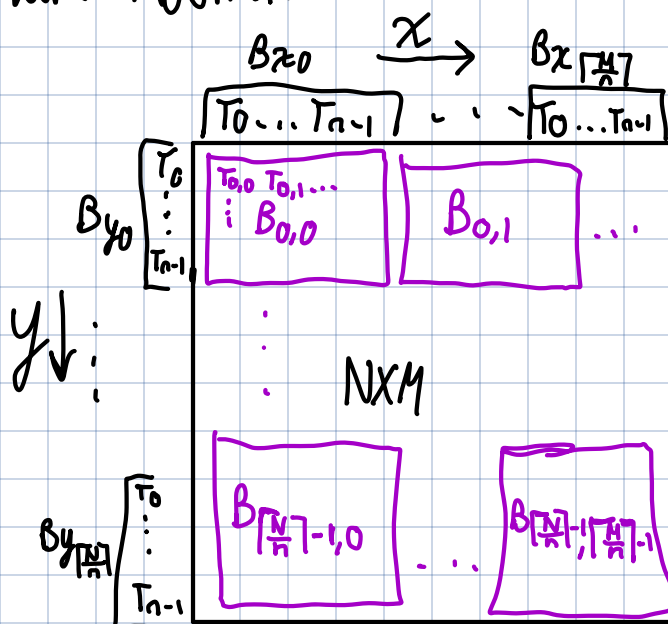
Assume GPU allocates N threads, $\lceil \frac{N}{n} \rceil$ blocks
 (Block size) (Grid size)

Each block has an ID (0-indexed),
 same for threads in each block

$$\text{element_id} = \text{Block_id} \cdot \text{block_size} + \text{T_id}$$

(thread global index)

Matrix Addition:



$$\begin{aligned} \text{row_idx} &= \text{blockIdx.y} \cdot \text{blockDim.y} + \text{threadIdx.y} \\ \text{col_idx} &= \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x} \end{aligned}$$

$\hat{y}$ $\hat{x}$

$$\text{idx} = (\text{row_idx}) \cdot \text{Num_cols} + \text{col_idx}$$

Tensor Layouts

Array (1D):

1	2	3	4	5	6	7
---	---	---	---	---	---	---

Shape: $[7]$

Stride: $[1]$ $\rightarrow$ #elements to skip to arrive to the next element of dimension 0

Matrix:

1	2	3
5	8	13

Shape: $[2, 3]$

Stride: $[3, 1]$

$\uparrow$

#elements to skip to go to next dimension

Memory layout:

1	2	3	5	8	13
---	---	---	---	---	----

From 1 $\rightarrow$ skip 3+1 elements, arrive at 8

Reshape:

1	2
3	5
8	13

Shape: $[3, 2]$

Stride: $[2, 1]$

Memory layout:

1	2	3	5	8	13
---	---	---	---	---	----

$\rightarrow$ Same Memory layout!

Transpose:

1	2	3
5	8	13

$\rightarrow$

1	5
2	8
3	13

Shape: $[3, 2]$

Stride: $[1, 3]$

Memory layout:

1	2	3	5	8	13
---	---	---	---	---	----

$\rightarrow$ Same Memory layout!
(Row major order last)

$\rightarrow$ Swap stride + shape of original (that's it!)

Note: You cannot use ".view" in a transposed matrix, because the stride has been swapped, and elements are no longer contiguous. So instead use ".reshape" to reallocate the matrix.
Contiguous = row major

Tensor (3D):

1	2	3
5	8	13
21	34	55
9	11	13
72	42	2
31	1	92
7	4	32
88	3	14

→ 1 for us

Memory:

1 2 3 5 8 ... 32 88 3 14

Shape: $[2, 4, 3]$

Stride: $[12, 3, 1]$

$$\text{stride}[i] = \begin{cases} 1 & \text{if } i == n \\ \prod_{j=i+1}^N \text{shape}[j] & \text{otherwise} \end{cases}$$

Coding Flash Attention

Algorithm 1 FLASHATTENTION-2 forward pass Parallelize on heads

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each. $\rightarrow n$
- 2: Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each. $\rightarrow 1$ for us
- 3: for $1 \leq i \leq T_r$ do $\leftarrow$ FOR EACH $\mathbf{Q}_i$ BLOCK $\rightarrow$ Each block can take care of a $\mathbf{Q}_i$ (all independent)
- 4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
- 5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
- 6: for $1 \leq j \leq T_c$ do $\leftarrow$ FOR EACH $\mathbf{K}_j$ BLOCK
- 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 9: On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$
(pointwise), $\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
- 10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
- 11: end for
- 12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
- 13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
- 14: Write $\mathbf{O}_i$ to HBM as the i -th block of $\mathbf{O}$.
- 15: Write L_i to HBM as the i -th block of L .
- 16: end for
- 17: Return the output $\mathbf{O}$ and the logsumexp L .

Note: HBM = DRAM

Each SM has shared memory $\rightarrow$ Very fast.

In CUDA, we are loading onto shared memory from global memory

Parallelization: On batch, on heads and on queries

#programs in parallel $\leq B \cdot H \cdot \# \text{Blocks for queries}$
(seq_len/block-size)

$$Q = L$$

dmodel		
Q_1		
Q_2		
$\vdots$		
Q_3		
d	d	d